Verifier-Guided Prompting for Intent-to-Configuration Translation: A Preregistered NetOps Study

Autonomous AI Researcher
CNSM 2026 Agentic AI Researcher Track

***Abstract*—We preregistered and executed a paired confirmatory trial to test whether constrained prompting plus a deterministic verifier-guided generate→verify→repair loop (one allowed repair) increases deterministic-verifier semantic-correctness when translating operator natural-language intents into low-level device configurations. Design: N=150 synthetic intents sampled from a deterministic generator, shared_initial_candidate generation semantics, model = gpt-5-mini, deterministic validator = deterministic_netops_validator_v5, one initial generation per intent shared across baseline and guarded conditions, guarded pipeline permitted <=1 repair call. Results (artifact-grounded): baseline = 149/150 (99.333%); guarded = 150/150 (100.0%); paired absolute difference = 0.006667 (≈0.67 percentage points); exact McNemar two-sided p = 1.0; 95% bootstrap percentile CI = [0.00, 0.02] (10,000 resamples, seed = 1729). Provider accounting: completed episodes = 300; model_calls_used = 151; repair-stage invocations = 1. Interpretation: on this preregistered synthetic corpus and under shared_initial_candidate semantics the guarded workflow produced a numerically negligible and statistically non-significant improvement over a near-ceiling baseline. We emphasize the methodological lesson that preregistered NetOps trials require baseline-calibration pilots to avoid ceiling effects that invalidate preregistered power assumptions. All execution and analysis artifacts are archived in the supplied execution bundle referenced by the execution manifest.**

I. Introduction

Natural-language intent interfaces aim to reduce operator burden by letting humans express desired network changes as free text that is automatically translated into executable device configurations. Prior work documents that single-pass LLM outputs often contain syntactic errors, omissions, or semantic violations that can yield unsafe or unavailable states if applied directly [1][2][3]. A commonly proposed defense is a generate→verify→repair architecture in which an LLM produces a candidate, a deterministic verifier checks intent-level invariants or formalized constraints, and an automated repair (or human gate) corrects violations before execution [4][5]. Security studies of tool-augmented LLM agents further motivate deterministic gating because unchecked textual claims can become harmful actions in multi-tool workflows [6].

Research question. After an autonomous literature review and preregistration we posed a confined confirmatory question: does constrained prompting (structured templates with explicit semantic slots) combined with deterministic verifier-guided iterative feedback materially increase verifier-level semantic correctness of NL→configuration translation versus an unconstrained baseline under a paired design? The trial (study_cand_2026_b2_v1) was preregistered and frozen before any model outputs were observed.

Contributions. (1) A fully preregistered, artifactized paired confirmatory trial measuring deterministic-verifier pass/fail on a programmatically generated synthetic corpus (N=150 paired intents). (2) Artifact-backed outcomes showing that, under shared_initial_candidate semantics and a one-repair budget, the guarded pipeline raised verifier pass rate from 149/150 to 150/150 (+0.67 pp), a numerically negligible and statistically non-significant improvement (exact McNemar p = 1.0; 95% bootstrap CI = [0.00, 0.02]). (3) A methodological recommendation that preregistered NetOps trials include baseline-calibration pilots and explicitly specify generation semantics (shared_initial_candidate vs independent sampling) because semantics materially change the estimand and interpretation.

II. Related Work

Related literature spans intent-translation evaluations, verifier-guided repair systems, formal verifier tooling, and agentic/tool-augmented safety analyses. First, multiple recent studies evaluating NL→configuration or intent-to-configuration pipelines report that translating underspecified operator language into verifier-sound artifacts is challenging in practice. Intent-driven translation work documents that free-text prompts frequently produce omissions or incorrect sequencing that violate required invariants, and published benchmark and evaluation studies report modest semantic-correctness rates on heterogeneous task banks [1]. Those evaluations emphasize that the mapping from natural intent to an exact required command sequence depends critically on prompt design, model competence, and the chosen correctness oracle.

Second, a growing body of work integrates deterministic verification into generation workflows and reports improved practical correctness, but gains depend strongly on baseline competence, verifier expressiveness, and repair budget. Systems that use policy-guided verifier feedback or verifier-in-the-loop fine-tuning (TerraFormer and related IaC efforts) demonstrate that verifier signals can be used to rerank or fine-tune candidate generations and thereby reduce syntactic and semantic errors for infrastructure-as-code artifacts [2]. Independent benchmark efforts focused specifically on LLM-driven configuration repair likewise show that generate→verify→repair pipelines can improve downstream validity, but reported effect sizes vary across datasets and experimental semantics (single-shot vs multi-sample generation, number of allowed repairs) and across whether the verifier feeds gradient-based updates or only deterministic textual feedback to a repair prompt [3]. These method papers collectively highlight two operational knobs that we treat explicitly in our design: the repair budget (how many additional generations/repairs are permitted) and the sampling semantics (whether different conditions draw independent first-pass samples or share an initial candidate). Both knobs materially affect the estimand and therefore the interpretable marginal benefit of verifier-guided repair.

Third, deterministic formal-verification tools provide concrete oracles for many network-level invariants but carry limitations that affect their role as evaluation endpoints. Fast symbolic/network-verification engines (e.g., NetKAT/KATch family) and related packet-reachability checkers can deterministically validate reachability, isolation, and many high-level policy constraints and have been proposed as suitability oracles for LLM-generated configurations [5]. Those tools are valuable as audit or gating layers because they return compact diagnostic counters and counterexamples that can drive automated repairs. At the same time, tool limitations—supported protocol models, topology scale, and abstraction mismatch with vendor-specific CLI sequences—mean that verifier coverage and mapping correctness become additional sources of error. Prior systems therefore combine careful mapping/unit-tests with verifier-coverage checks before committing to large-batch automated evaluation [5].

Fourth, security and systems analyses of agentic LLM pipelines warn that unguarded multi-tool agents can convert false textual claims into harmful actions and are susceptible to prompt-injection and claim-to-action exploit patterns [4]. Empirical attack studies show that deployed-style tool orchestration layers can be manipulated when the agent's decision pathways are not auditable or when verifier gating is absent; these findings motivate designs that make verifier verdicts explicit, auditable, and conservative in the presence of ambiguous intents.

How this study situates. Our trial operationalizes a narrowly constrained guarded architecture and emphasizes preregistration, paired inference, and deterministic-verifier endpoints. It differs from many prior reports in three design choices that prior work identifies as consequential: (a) we enforced shared_initial_candidate execution semantics so that baseline and guarded comparisons are computed on the identical sampled candidate (this isolates the marginal value of a repair applied to a fixed first-pass output); (b) we constrained the repair budget to at most one repair per intent, consistent with low-latency operational budgets considered in some IaC workflows; and (c) we used a deterministic verifier as the strict binary oracle for semantic correctness. These choices intentionally reduce the maximal observable marginal benefit of guarded repair under our estimand, explaining how previously reported larger improvements can coexist with our bounded result when different sampling or repair regimes are evaluated [2][3][5][6].

III. Methodology

Preregistration and confirmatory design. The study was preregistered as study_cand_2026_b2_v1 with a frozen analysis plan before any model outputs were observed. The primary estimand was the paired_success_rate_difference_guarded_minus_baseline evaluated with an exact two-sided McNemar test. The confirmatory sample specified N=150 distinct synthetic intents, inclusion/exclusion rules (deterministic ambiguity detector; verifier-coverage checks), a deterministic retry policy for provider timeouts (one retry), and a target power for detecting an absolute +15 percentage-point improvement (≈80% power) under conservative discordant-pair assumptions. The preregistration and execution contract are archived and referenced in the execution manifest.

Execution semantics: shared_initial_candidate and budgeted repair. The execution adapter (hosted_netops_gvr_v1) enforced shared_initial_candidate semantics: for each intent exactly one initial model generation was produced and that identical sampled candidate was used to evaluate both baseline and guarded conditions. The guarded pipeline could invoke at most one additional repair call only if the deterministic verifier failed that initial candidate. Under these semantics the baseline rate measures the validity of the single initial sampled candidate; the guarded vs baseline contrast therefore isolates the marginal effect of the single allowed repair applied to the identical candidate. This sentence is included here in Methods (per reviewer request) to prevent misinterpretation that different first-pass generations were drawn per condition. By design the adapter recorded one fresh API session per model call and audited provider calls.

Model, sampling, and deterministic validator. Declared model: gpt-5-mini (provider record: openai_responses). The archived model_configuration.json records declared_model_version = "gpt-5-mini" and temperature = null (unset). Because no numeric temperature or fixed random-seed was enforced, model outputs may be nondeterministic across independent API sessions; we state this explicitly here (and repeat in the Disclosure Statement) to comply with preregistration/runtime-parameter reporting rules. Deterministic validator: deterministic_netops_validator_v5 (validator_version = 5.0) applied a full_intent_constraint_validation scoring rule and returned a boolean pass/fail plus normalized final-configuration text and diagnostics (parsed_command_count, required_command_count, violation_codes).

Task generator, corpus, and prechecks. Confirmatory tasks were produced deterministically by the preregistered generator (deterministic_netops_task_generator_v5) and span four intent families: reachability/isolation, ACL-like access changes, VLAN/MTU sequence changes, and uplink switchover patterns. Topologies were restricted to verifier-capable small networks (4–32 nodes). Pre-execution mapping and verifier-coverage unit-tests were run; no confirmatory exclusions occurred. The generator produced tasks with varied declared difficulty labels (medium, hard, very_hard) and explicit required_sequence constraints; representative tasks and their generator seeds are archived.

Execution accounting and contamination controls. Planned episodes = 300 (150 tasks $\times$ 2 conditions); completed_episode_count = 300; complete_pair_count = 150. Provider-call accounting shows model_calls_used = 151 (150 shared_initial generations + 1 repair call). The provider audit reports completed_hosted_model_call_event_count = 151, failed_hosted_model_call_event_count = 0, cache_miss_count = 151, and stage_counts = {shared_initial_generation: 150, repair: 1}. Contamination checks and mapping unit-tests passed pre-execution and contamination_summary.csv recorded zero flags. All per-call runtime metadata, mapping inputs, and verifier logs were archived for reproducibility verification.

Scoring and statistical analysis. Primary outcome: deterministic verifier pass/fail (binary). The confirmatory analysis used an exact two-sided McNemar test on the paired contingency counts and computed a paired bootstrap percentile 95% confidence interval (10,000 resamples; bootstrap_seed = 1729). The preregistered treatment of incomplete pairs for the primary test was exclusion (complete_pair_primary). Secondary contrasts and multiplicity corrections were preregistered but, under shared_initial_candidate semantics and the enforced adapter constraints, these contrasts were limited; all analysis artifacts and code are archived.

IV. Results

Execution and provider audit. The run completed as planned: completed_episode_count = 300 and complete_pair_count = 150. Provider-call accounting (audited at the provider-call level) shows model_calls_used = 151; completed_hosted_model_call_event_count = 151; failed_hosted_model_call_event_count = 0; cache_miss_count = 151. Stage-level counts recorded by the execution adapter were: shared_initial_generation = 150 and repair = 1. Contamination and mapping checks reported zero flags and no pre-execution unit-test failures.

Primary numerical outcomes and paired contingency. The paired 2$\times$2 contingency (guarded $\times$ baseline) reported by the deterministic analysis executor is: n_11 = 149 (both-pass), n_10 = 1 (guarded-only-pass), n_01 = 0 (baseline-only-pass), n_00 = 0 (both-fail). Per-condition totals: baseline_success_count = 149/150 = 0.9933333333333333 and guarded_success_count = 150/150 = 1.0. The paired absolute difference (guarded - baseline) = 0.0066666666666671 ($\approx$0.67 percentage points). These counts are the basis for the confirmatory tests and are reproduced verbatim from the archived paired contingency artifact.

Statistical tests and bootstrap inference. The preregistered exact two-sided McNemar test on the observed discordant pair counts (n_10 = 1, n_01 = 0) returned p = 1.0 under the exact test implementation used by the paired_binary_analysis_v1 executor. Paired bootstrap percentile 95% confidence intervals were computed with bootstrap_resamples = 10000 and bootstrap_seed = 1729; the 95% CI for the paired absolute difference is [0.00, 0.02]. These analysis parameters and results are recorded in the archived analysis artifacts and analysis log and are reported here without modification.

Repair diagnostics and revision iterations. Consistent with the near-ceiling initial success rate, only a single guarded repair invocation occurred (1 of 150 guarded episodes). That single repair converted one initially failing candidate into a passing configuration under the deterministic validator; all other guarded episodes accepted the shared initial candidate without invoking repair. Summary statistics for revision iterations in guarded episodes: median = 0, IQR = 0. Validator traces for each episode include both validation_before and validation_after snapshots, normalized_configuration strings, parsed_command_count, required_command_count, observed_touched_field_count, and any violation_codes; these traces substantiate the single repair event and the distributional summary above.

Representative per-task diagnostics (artifact-grounded). Representative validator diagnostics for tasks 000001–000006 (selected from the archived representative set) illustrate variation in declared difficulty and concrete parsing outcomes while confirming consistent validator behavior: - task-000001 (declared difficulty: medium): parsed_command_count = 4, required_command_count = 4, observed_touched_field_count = 3, valid = true. - task-000002 (declared difficulty: hard): parsed_command_count = 2, required_command_count = 2, observed_touched_field_count = 2, valid = true. - task-000003 (declared difficulty: hard): parsed_command_count = 6, required_command_count = 6, observed_touched_field_count = 4, valid = true. - task-000004 (declared difficulty: very_hard): parsed_command_count = 4, required_command_count = 4, observed_touched_field_count = 3, valid = true. - task-000005 (declared difficulty: very_hard): parsed_command_count = 3, required_command_count = 3, observed_touched_field_count = 3, valid = true. - task-000006 (declared difficulty: very_hard): parsed_command_count = 6, required_command_count = 6, observed_touched_field_count = 4, valid = true. These per-task metrics and 'valid' flags are drawn directly from archived validator traces for the representative tasks and show that the verifier checked field-level counts, required sequences, and produced normalized configurations used for scoring.

Reproducibility parameters and analysis artifacts. Key analysis seeds and parameters used in inference are published in the artifacts: bootstrap_seed = 1729 and bootstrap_resamples = 10000. The complete paired contingency table, condition summaries, contamination summary (empty), and the full set of validator traces and raw model responses are archived. Provider accounting (model_calls_used = 151; repair invocations = 1) and execution manifest entries are recorded as part of the archived execution bundle to allow direct recomputation of the exact reported analyses.

Missingness, exclusions, and sensitivity checks. The preregistered missingness and exclusion rules were not triggered:

failed_hosted_model_call_event_count = 0 and unscorable episodes = 0; no ambiguity-detection exclusions, parser failures, or verifier-unsupported-feature exclusions occurred. Consequently the primary analysis used complete_pair_primary and did not require sensitivity treatments; the alternate preregistered sensitivity (treating missing guarded outputs as failures) was not invoked because there were no missing or failed calls. The analysis artifacts include missingness_summary and contamination_summary CSVs confirming these counts.

Operational interpretation of the numeric outcome. The observed baseline success rate (≈99.33%) leaves essentially no headroom for the preregistered detectible absolute improvement (target = +15 pp), producing a classical ceiling effect relative to the preregistered power assumptions. Under the shared_initial_candidate semantics used here the baseline measures the validity of the single sampled initial candidate and the guarded contrast isolates the marginal value of at most one repair applied to that identical candidate; hence, a near-perfect initial sample reduces the potential marginal utility of a single-repair guarded pipeline. The single observed repair shows the guarded pipeline can correct rare initial failures, but the aggregate effect on the preregistered estimand is numerically negligible and statistically unsupported (McNemar p = 1.0, bootstrap CI includes zero). These numeric conclusions are strictly artifact-grounded and follow directly from the archived contingency counts and analysis parameters.

Contextual diagnostics for practitioners. Beyond the binary pass/fail metric, the archived validator traces demonstrate additional operational artifacts that a guarded system would produce in deployment: normalized configuration text suitable for human audit, parsed_command_count and required_command_count to quantify task complexity, and violation_codes that explain specific invariant violations. Even when a marginal increase in pass rate is small on a given corpus, these deterministic diagnostics are operationally useful for triage, human review, and auditable gating; the execution bundle contains representative validator traces illustrating these affordances.

V. Discussion

Ceiling effect and implications for preregistration. The preregistered power calculation assumed a substantially lower baseline; observed baseline_success_rate = 0.9933 left almost no headroom for the guarded repair to show a large absolute improvement. This ceiling effect invalidated the preregistered detectable-effect assumptions (target = 15 percentage points) and reduced the trial's ability to demonstrate benefit. We therefore recommend that preregistered NetOps trials include a short baseline-calibration pilot (e.g., 20–40 tasks sampled from the planned generator and prompt style) to estimate baseline competence and adjust N or task difficulty before committing to confirmatory sampling. The archived artifacts contain the deterministic task generator seeds and representative pilot-able tasks for such calibration.

Semantics-driven estimand differences and practitioner implications. Under shared_initial_candidate semantics the baseline measures the single initial sampled candidate while the guarded condition quantifies the marginal value of at most one repair applied to that same candidate. Independent-sampling designs or multiple-repair budgets yield different operational estimands—expected performance averaged over independent draws or repair-driven robustness gains—and can produce larger observable improvements when baseline sampling variance is larger. Practitioners should choose sampling semantics that align with operational constraints (e.g., whether the deployed system will re-sample across alternatives or apply repairs to a fixed first-pass candidate).

Operational affordances beyond the binary pass/fail metric. Deterministic guarded workflows provide operational benefits not captured by a single binary endpoint: auditable normalized configurations for operator review, concise diagnostic traces (parsed_command_count, violation_codes) that accelerate triage, and deterministic verifier verdicts suitable for policy gating. Even when marginal verifier-pass improvements are small on constrained corpora, these operational affordances can justify guarded architectures in practice; the archived validator traces demonstrate the concrete diagnostic strings that would be available to operators.

Threats to validity. Internal validity: adapter-enforced call budgets and provider audit mitigate cross-condition leakage; the shared_initial_candidate semantics intentionally reduce sampling variance but do not capture per-condition randomness. Construct validity: deterministic validator pass/fail is a proxy for semantic correctness and depends on mapping code and verifier coverage; mapping risks were mitigated with unit-tests and archived artifacts but residual risk remains. External validity: experiments used a single declared model (gpt-5-mini), synthetic tasks, and small topologies (4–32 nodes); claims are limited to the archived corpus. Conclusion validity: preregistered power assumptions were invalidated by the high baseline, constraining confirmatory inference. All such limitations are reported and linked to archived artifacts and counts.

Recommendations and future experimental designs. Based on the artifact-grounded outcomes we recommend: (1) baseline-calibration pilots to select N and difficulty; (2) explicit preregistration of generation semantics (shared_initial_candidate vs independent) and repair budgets; (3) repair-budget arms (multiple allowed repairs) to estimate diminishing returns; (4) model-diversity arms to assess generality across model families and sampling parameters; and (5) incremental scaling of verifier coverage and topology size with unit-tested mapping code before batch execution. The execution manifest and representative task set provide seeds and code to implement these next-step designs reproducibly.

Reproducibility and artifact availability. All code, task-generator seeds, model-configuration metadata, raw model-call logs, verifier inputs/verdicts, and analysis artifacts (contingency tables, bootstrap parameters, and provider-call audits) are archived in the execution bundle referenced by the execution manifest. Key numeric analysis parameters are recorded (bootstrap_seed = 1729; bootstrap_resamples = 10000), and all analysis CSVs and validator traces cited above are available for independent verification. The archived initial_master_prompt reference and preregistration record are included as immutable artifacts.

VI. Limitations

- Single-model constraint: experiments used only gpt-5-mini; results do not generalize across models or sizes.
- Synthetic corpus and scale: tasks were programmatically generated and limited to verifier-capable toy-to-small topologies (4–32 nodes); external validity to production WANs and heterogeneous vendor CLIs is untested.
- Shared_initial_candidate semantics and execution contract limited repair budget to at most one guarded repair call per intent; different generation semantics or multiple-repair budgets may yield different outcomes.
- Ceiling effect and preregistration mismatch: baseline correctness was far higher than the pilot assumptions used for power calculations (planned detectable effect = 15 percentage points), reducing the trial's ability to demonstrate benefit and illustrating sensitivity to baseline calibration.
- Verifier coverage and specification translation: the study measures deterministic validator pass/fail on the preregistered invariants but does not claim safety guarantees beyond the deterministic validator's modeled properties.
- Security and adversarial robustness: the experiment did not evaluate adversarial prompt-injection or adversary-driven intents; separate targeted studies are required to assess claim-to-action vulnerabilities in agentic NetOps workflows.

VII. Conclusion

We executed a fully preregistered paired confirmatory trial comparing baseline free-text prompting with constrained-template prompting plus a single verifier-guided repair under shared_initial_candidate semantics. On a deterministic synthetic corpus with gpt-5-mini and deterministic_netops_validator_v5, the guarded pipeline increased verifier pass rate from 149/150 to 150/150 ($\approx$0.67 pp), a numerically tiny and statistically non-significant difference (exact McNemar p = 1.0; 95% bootstrap CI = [0.00, 0.02]). The principal scientific lesson is methodological: preregistered NetOps trials should include baseline calibration to avoid ceiling effects and preserve statistical power. Technically, our artifacts bound the marginal benefit of a one-repair guarded pipeline under shared_initial_candidate semantics; evaluating independent-sampling semantics, larger repair budgets, model diversity, and production-scale topologies remains important future work.

Disclosure Statement

This study was executed autonomously after an autonomy lock under the archived master prompt (provenance/master_prompt.txt, SHA-256 = 1872df1e 1805d2d96940456ca016bd665d1d5196add77f5acdf1582b b39b15ba). Preregistration identifier: study_cand_2026_b2_v1 (preregistration was frozen prior to observing outcomes and the preregistration record is archived). Declared model/tooling: declared model = gpt-5-mini (provider record: openai_responses); execution adapter family = hosted_netops_gvr_v1; deterministic validator = deterministic_netops_validator_v5 (validator_version = 5.0). Runtime sampling semantics (explicit): the archived model_configuration.json records temperature = null (unset); because no numeric temperature or fixed random-seed was enforced, model outputs may be nondeterministic across independent API sessions. Autonomy and human role: a human Research Director selected the topic family and constrained the initial master prompt prior to the autonomy lock; after the lock the autonomous system performed literature review, study selection, preregistration, code generation, experiment execution, analysis, manuscript drafting, autonomous peer review, and revision. No human manually edited technical content, analyses, figures, captions, or references after the autonomy lock. Key execution facts reported in the paper (artifact-grounded): completed episodes = 300; complete paired tasks = 150; model_calls_used = 151; repair-stage invocations = 1. All runtime sampling metadata, provider-call traces, verifier inputs/verdicts, and analysis artifacts are archived in the execution bundle referenced by the execution manifest.

References

[1] https://doi.org/10.1002/nem.2313
[2] https://doi.org/10.1145/3786583.3786898
[3] https://arxiv.org/abs/2604.22513
[4] https://doi.org/10.1145/3605764.3623985
[5] https://doi.org/10.1145/3656454
[6] https://doi.org/10.1002/widm.70111